

Disagreement as Evidence: Predicting Blind Spots in Agent-Skill Security Scanners

Abstract

Automated security analyses routinely disagree, yet evaluations treat that disagreement as a statistic (Cohen’s κ , pairwise overlap) and stop there, as if it were noise or merely an outcome about tool quality. We propose to treat disagreement differently: as *analytical evidence about the analyses themselves*. The structure of disagreement, once made comparable, exposes each analysis’s coverage of the threat space and how it operationalizes detection there. We develop a methodology for this conversion: a shared observation language that makes disagreement localizable; a diagnostic decomposition that attributes a miss to the earliest demonstrated failure in *coverage* (the input was never collected), *detection* (the engine did not recognize it), or *decision* (the finding was discarded); blind-spot hypotheses locked before construction; and adversarial validation. We instantiate the idea on agent-skill security scanners, an emerging supply-chain defense whose scanners disagree sharply. Across 581 malicious skills (350 wild, 231 coordinate-generated) and three heterogeneous scanners, the decomposition predicts concrete blind spots, and locked adversarial construction confirms them: hidden-file payloads evade SkillSpector on 14/15, finding-family variants evade Cisco on 6/6, and combined strategies evade SkillSpector on 5/5, with selected failures traced to source-level mechanisms. Disagreement, we argue, is not merely a statistic to report: it is evidence to mine.

Keywords

agent skills, security scanners, disagreement analysis, blind spots, adversarial construction

1 Introduction

Automated security analyses disagree. Given the same artifact, one scanner flags it, another passes it, and a third flags something else entirely. The standard response is to *measure* the disagreement (Cohen’s κ , pairwise overlap, per-tool recall) and treat the result as an evaluation outcome: a fact about tool quality, or noise to be averaged away. We ask a different question: *what does the pattern of disagreement reveal about the analyses themselves?* We argue that disagreement is not merely a statistic to report. It is analytical evidence: once disagreements are made comparable, their structure exposes what each analysis covers, what it cannot see, and why.

Today this evidence is inaccessible, because analyses speak incompatible languages. One scanner reports “tool poisoning” another “supply-chain risk,” a third a family of regex matches. There is no shared frame in which to ask *where exactly* they disagree, so disagreement remains global (a κ) but never localizable. Without localization there is no explanation; without explanation, disagreement cannot be acted upon.

We convert disagreement into structure in four steps: map scanner categories into a shared observation language; attribute each diagnosable miss to the earliest demonstrated failure in *coverage*, *detection*, or *decision*; lock the resulting blind-spot hypothesis; then

test it by adversarial construction and trace selected failures to implementation mechanisms.

We instantiate this idea on *agent-skill security scanners*. Coding agents extend their capabilities by installing skills (packages of instructions and scripts), and malicious skills are a live supply-chain threat (the ClawHavoc incident surfaced over 1,184 of them [6]). Scanners have emerged to screen skills, but they disagree sharply [2, 6], and no existing evaluation explains the disagreement. The object is timely; the methodology is the contribution.

An honesty point frames our results. In aggregate the scanners are strong: across 581 malicious skills, SkillSpector flags 92.9%, Cisco 86.7%, Caterpillar 72.6% under a uniform reading of their outputs (any surfaced finding counts; operationalized in Table 5). The blind spots are therefore invisible in average recall; they emerge only when the threat space is partitioned. This is precisely the structure that aggregate metrics conceal, and precisely what disagreement, treated as evidence, exposes.

Contributions. Three, in strict hierarchy:

- **Core idea.** Disagreement among automated security analyses is not merely an evaluation outcome: projected into a shared observation space, its structure becomes evidence about the analyses themselves, from which falsifiable blind-spot predictions can be derived.
- **Methodology.** A loop converting disagreement into such predictions: observe where scanners disagree, decompose why (coverage, detection, decision), lock what they will miss next, and test whether the prediction holds.
- **Demonstration.** An end-to-end instantiation on agent-skill security scanners: 581 malicious skills, 500 benign references, three scanners, five locked predictions plus a stability control, with confirmed, partial, and failed outcomes traced to mechanisms.

Scope. We demonstrate the loop end-to-end on one instantiation. The generality of the idea (package scanners, dependency scanners, static analyzers, LLM-based security evaluators) is the research direction this early result opens, to be developed in an extended version.

Results. Locked predictions anticipated hidden-file evasion of SkillSpector (14/15), regex blindness to semantic phrasing (10/10), and Cisco’s failure on finding-family variants (6/6); another prediction was partial, while a stability control failed. Each confirmed prediction traces to observable intermediate evidence or a named mechanism. The bypasses are not the contribution; prospective diagnosis is.

2 The Core Idea: Disagreement as Evidence

Central proposition. *Once heterogeneous scanner outputs are semantically aligned and their intermediate states are observable, localized disagreement can predict scanner-specific failures on previously unseen artifacts.* The proposition is falsified if the diagnosis cannot be reproduced from the recorded evidence or if its locked

Table 1: What each approach can do with disagreement. “Localize” = say where in the threat space the disagreement arises; “explain” = attribute it to a mechanism in the analysis; “predict” = derive the failure before any new artifact is built.

	Localize	Explain	Predict
Aggregate recall	×	×	×
Pairwise κ / overlap	×	×	×
Manual post-hoc audit	✓	✓	×
Our loop	✓	✓	✓ (locked)

prediction fails on a new construction. It is stronger than observing another bypass: the failure family and scanner must be named before the test artifact exists.

An automated security analysis is a pipeline: it *collects* an input, *detects* patterns in it, and *decides* what to report. The stages fail differently across analyses. This gives the paper’s central object an operational shape:

Three-layer diagnostic rule. We assign a miss to the earliest *demonstrated* failing layer: *L1 coverage* if scanner traces show that the relevant input was not collected; *L2 detection* if the input was collected but no relevant finding was produced; and *L3 decision* if such a finding was produced but the verdict discarded it. If intermediate evidence is unavailable, the layer is unresolved rather than inferred from the final verdict. Multiple layers may be weak on one artifact; earliest-failure attribution identifies the first sufficient cause, not a unique latent cause.

The layers are separable when the required intermediate evidence is exposed. A payload placed in a file the collector skips establishes L1 regardless of engine strength. Collected input with no corresponding finding establishes L2. A finding demoted below a severity threshold establishes L3. Each attribution implies a different fix (collect more inputs; change engines; change thresholds), attack strategy, and falsifiable blind-spot hypothesis.

The paper’s progression is the same ladder: the observation space answers *where* scanners disagree (§3), the three layers answer *why* (§5), locked hypotheses answer *what they will miss next* (§6), and construction answers *whether the prediction is true* (§7). Why is a new instrument needed to climb it? Because the standard tools operate at the wrong level of aggregation (Table 1):

Recall and κ compress disagreement to a scalar; scalar differences cannot say *where* the analyses diverge. A manual audit can, after seeing the failures; but post-hoc explanation cannot distinguish prediction from rationalization. The loop we instantiate (observe, localize, decompose by layer, hypothesize, lock, construct, validate) is the minimal pipeline that does all three, and its predictions are checkable against a locking protocol that fixes hypotheses before construction.

3 An Observation Instrument

The reason disagreement cannot currently be interrogated is that the scanners speak incompatible languages: SkillSpector speaks of 17 risk classes, Cisco of 18 threat categories, Caterpillar of regex rule families. There is no frame in which to ask *where exactly* they

Table 2: The three-dimensional observation language. Each threat is a coordinate (*source, mechanism, target*); all three axes are multi-valued.

Dimension	Question	Values (abbreviated)
source	where does it enter?	supply chain; user input; external untrusted content; runtime environment; unknown
mechanism	how does it attack?	code execution; instruction manipulation; dependency manipulation; privilege abuse; obfuscation; state corruption; ...
target	what does it seek?	information theft; persistent control; resource abuse; system damage; financial theft; content-safety harm; defense evasion

disagree. The coordinate space we introduce is a *shared observation language*, not a universal taxonomy. Figure 1 shows the conversion. A running coordinate makes it concrete: (*supply chain, code execution, information theft*) aligns credential-exfiltration categories across scanners. SkillSpector recognizes script semantics there, Cisco emphasizes literal triggers and severity, and Caterpillar sees only lexical markers. Their disagreement at one coordinate therefore suggests distinct no-literal, hidden-placement, and threshold predictions, which §7 tests.

3.1 Three Dimensions

Each threat is located by three dimensions (Table 2): *source* (where it enters: supply chain, user input, external untrusted content, runtime environment, or unknown), *mechanism* (how it attacks: code execution, instruction manipulation, dependency manipulation, privilege abuse, obfuscation, state corruption, ...), and *target* (what it seeks: information theft, persistent control, resource abuse, system damage, financial theft, content-safety harm, defense evasion). The axes follow established security modeling: the attacker model (cf. Dolev–Yao), the technique axis (cf. ATT&CK), and the security-property axis (cf. the CIA triad).

3.2 Mapping Scanner Languages

We mapped the documented category systems of six scanners (SkillSpector, Cisco, Snyk, Socket, ATH, AARTS; 168 threat categories) into 43 occupied coordinates. Each native category is assigned to the coordinates its triggering conditions observe, with multi-valued assignment when it spans several coordinates. Two categories whose coordinate sets overlap are nominally aligned; residual disagreement can then be investigated as operational rather than merely terminological. This mapping is currently evidence-based but single-annotator: it supports the present feasibility claim, not a claim that the observation language is already a reliable community taxonomy. Independent annotation and agreement measurement are explicit requirements for the extended benchmark.

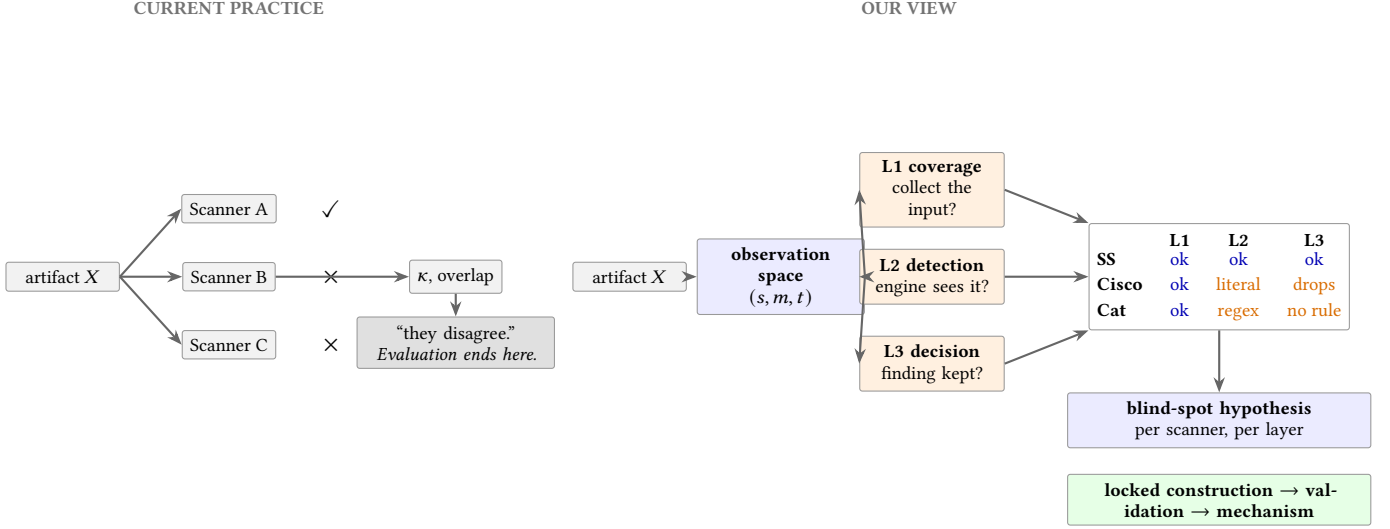


Figure 1: The same disagreement, read two ways. Current practice collapses three verdicts on artifact X into a scalar and stops. Our view localizes X in a shared observation space, decomposes each scanner’s failure into one architectural layer (coverage L1, detection L2, or decision L3), derives per-scanner blind-spot hypotheses, and validates them by construction locked before the fact.

3.3 Orthogonality

The dimensions carry complementary information: on an 81-category human-verified gold standard, pairwise normalized mutual information stays moderate (0.23–0.57), no axis is redundant (removing any one collapses its 31 occupied coordinates to 15–23), and its 81 native labels compress into 31 coordinates (2.6×) as synonymous scanner vocabularies merge while distinctions survive. The point of the space is not taxonomic completeness; its role here is *inferential*: once scanner outputs are projected into a common coordinate system, disagreement becomes evidence from which hypotheses about missing coverage and operational boundaries can be derived, the input to everything that follows.

4 Instantiation: Agent-Skill Scanners

Agent skills are distributed through marketplaces such as ClawHub and skills.sh [6]; at scale, a snapshot of 138,133 public SKILL.md files found 91.8% exhibiting at least one packaging or safety defect [12], and field measurements attribute 26.1% of 31,132 wild skills with at least one security vulnerability [7]. Because a skill can carry arbitrary instructions and scripts, it is an active supply-chain attack surface: the ClawHavoc incident identified over 1,184 malicious skills in a single marketplace [6], and injected skill instructions achieve up to 80% attack success on frontier agents [9]. Security scanners have emerged to screen skills before installation [2].

We study three scanners spanning distinct detection architectures (Table 3). **SkillSpector** combines static rules, AST analysis, YARA signatures, and an LLM semantic layer. **Cisco Skill Scanner** combines static analysis, YARA, LLM-based judgment, and behavioral dataflow. **Caterpillar** is a lightweight regex-based tool. The first two expose category systems and can be projected into the observation space; Caterpillar has no comparable semantic categories

pairwise κ (136 skills × 6 scanners)

SS	1.00	-.01	.03	.24	.02	.05
VT	-.01	1.00	.11	.01	.12	.09
Static	.03	.11	1.00	.01	.11	.20
Snyk	.24	.01	.01	1.00	.11	.16
Socket	.02	.12	.11	.11	1.00	.21
ATH	.05	.09	.20	.16	.21	1.00
	SS	VT	Static	Snyk	Socket	ATH

Figure 2: Disagreement is the norm in the wild. A market census (project week-0 data, no ground truth; its six scanners overlap with \$5’s three only in SkillSpector): on 136 skills verified across platforms, every pair of public scanners agrees at $\kappa \leq 0.244$, most near zero. Consensus is equally scarce: only 5.9% of skills are flagged by four or more scanners, and on the full 1,082-skill census 260 of 445 flagged skills are flagged by exactly one. Disagreement at this scale is not noise to average away; it is the ecosystem’s default output.

and therefore participates only as an operational lower-bound reference. Its inability to be projected is itself a boundary condition of our method, not evidence of coordinate-level absence.

Table 3: The three studied scanners. “Decision rule” is what the raw output reduces to for an allow/block decision: each rule is later implicated in a distinct blind spot.

Scanner	Layers	Decision rule
SkillSpector	static rules, AST, YARA, LLM semantic	risk score (0–100); >50 = detected
Cisco	static, YARA, LLM judgment, dataflow	safe unless a CRITICAL/HIGH finding exists
Caterpillar	regex families	letter grade; $\neq A$ = flagged

Table 4: Four corpora, four roles. Each row is a distinct evidential unit; no number crosses rows. Census scanners: SkillSpector, Snyc, Socket, ATH, VirusTotal, static analysis. Main-loop scanners: the three of Table 3.

Corpus	<i>n</i>	Role in this paper
Market census (wild, unlabeled)	1,082	Motivation: disagreement is pervasive (Fig. 2)
Malicious corpus	581	Observed disagreement: 350 wild + 231 generated by occupied coordinate
Benign references	500 + 4,000	False-positive mirror (500 in-corpus; 4,000 official MalSkill-Bench audit)
Constructions	129 (79 conf.)	Prediction validation: Arms 7–13 locked; 1–6 exploratory; disjoint from the corpus above

Threat model. The attacker authors and publishes a skill that a user (or an agent acting for the user) installs from a marketplace. The skill may contain natural-language instructions in SKILL.md, arbitrary scripts, and metadata declarations; everything is readable, but nothing is executed at install time. The defender runs one or more scanners over the skill *before* execution and must decide allow/block with no runtime evidence. The attacker’s goal is not to exploit a parser bug but to place malicious behavior where the defender’s analysis does not look: outside the scanned file set, outside the literal command surface, or inside a declared capability. All blind spots we report are of this placement kind: architectural, not cryptographic.

These scanners frequently disagree on the same skill. Recent measurements quantify the disagreement: across 67,453 skills, pairwise overlap of flagged sets is at most 10.4%, and only 0.69% are flagged by all scanners [6]. Yet such numbers stay descriptive: they tell us *that* scanners disagree, not *where* in the attack space the disagreement arises or *why*.

5 Observed Disagreement, Layer by Layer

Phase A: observe and localize. With a common observation layer in place, we project the two category-bearing scanners onto the coordinate space and compare all three operationally. We measure 350 wild malicious samples stratified by behavior and 231 coordinate-generated samples, reported separately before aggregation, plus 500 unaudited benign marketplace references. Generated samples

Table 5: Observed disagreement under one reproducible decision rule fixed before analysis: Cisco flagged iff `is_safe=false` or any MEDIUM-or-higher finding; SkillSpector iff risk score > 0; Caterpillar iff any finding. Cisco returned null verdicts on 17 malicious skills (counted as non-detections); SkillSpector completed all 581. “Shipped rules” rows show what the same raw outputs say under each product’s own threshold (Cisco CRITICAL/HIGH; SkillSpector score > 50; Caterpillar grade $\neq A$). Benign reference: 500 marketplace skills, unaudited; flag rates are upper bounds on false positives.

<i>(a) Detection rate by subset</i>			
Subset	Cisco	SkillSpect.	Caterp.
Malicious (581), uniform rule	86.7%	92.9%	72.6%
same outputs, shipped rules	71.6%	46.8%	38.6%
Benign ref. (500), uniform rule	35.2%	44.2%	28.6%
same outputs, shipped rules	11.2%	2.8%	16.4%

<i>(b) Pairwise consensus (third scanner is the sole miss)</i>			
Pair flags	<i>n</i>	sole miss	
Cisco + SkillSpector	118	Caterpillar	
SkillSpector + Caterpillar	40	Cisco	
Cisco + Caterpillar	17	SkillSpector	
Single-scanner flags	39	SS 23 / Cisco 10 / Cat 6	

are specification-checked for the intended behavior but not runtime-verified; their results therefore establish controlled detectability, not ecological prevalence. Likewise, benign flag rates are upper bounds, not false-positive rates.

5.1 Aggregate Detection Conceals the Structure

Table 5 shows what the aggregate view sees, and what it hides. Under the uniform reading, SkillSpector flags 92.9%, Cisco 86.7%, Caterpillar 72.6% of the 581 malicious skills (wild 350: 91.1/88.0/70.9; generated 231: 95.7/84.8/75.3, per-class detail in Fig. 3c). Under each product’s *shipped* decision rule the same raw outputs yield 46.8%, 71.6%, and 38.6%: the disagreement statistics are functions of thresholds, not only of scanners. Stopping at any of these numbers, one concludes detection is largely solved or largely broken depending on the rule chosen. The structure emerges one level down (Fig. 3b): 222 skills (38.2%) are missed by at least one scanner, and when two agree the third is the sole miss (Caterpillar 118, Cisco 40, SkillSpector 17); and across the 13 wild behavior classes, per-class detection spans 0–100% (Fig. 4).

5.2 The Statistic Itself Is Unstable

Figure 3a computes pairwise Cohen’s κ from the *same* 581×3 raw outputs under three defensible choices: malicious-only with shipped rules (−0.01 to +0.24), malicious-only with the uniform rule (+0.08 to +0.16), malicious-plus-benign with the uniform rule (+0.29 to +0.44). The benign corpus dominates the third condition because agreement on easy negatives inflates κ ; the shipped-rule condition collapses because thresholds disagree more than detections do. No scalar in this range says *where* the scanners differ or *why*; the rest of the paper supplies the missing structure.

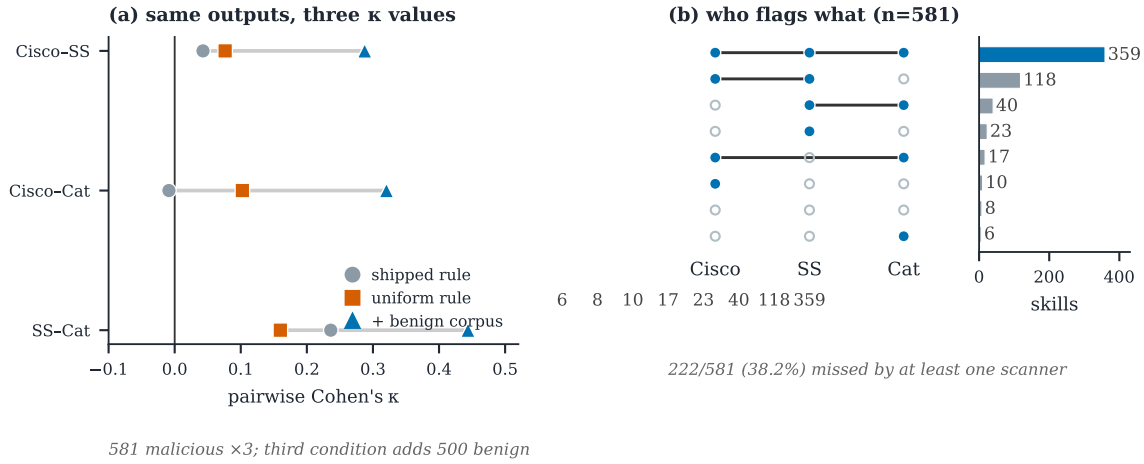


Figure 3: The aggregate view, dismantled. (a) Pairwise Cohen’s κ from the *same* 581 $\times$ 3 raw outputs under three corpus/rule choices: the statistic swings from -0.01 to $+0.44$; no scalar in this range localizes the disagreement. (b) Flag combinations: misses concentrate in reproducible patterns (Caterpillar sole miss: 118), not noise. Where those misses live in the threat space is Fig. 4’s question.

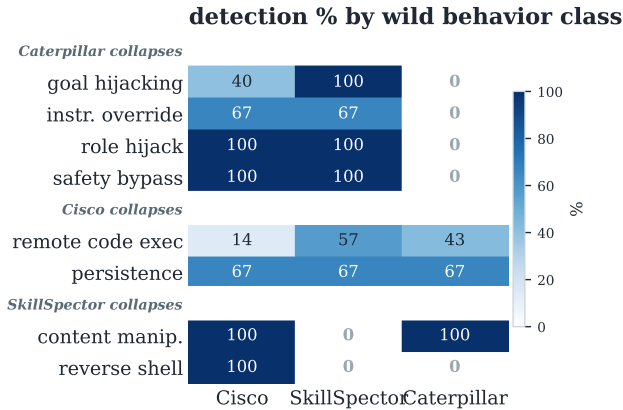


Figure 4: Localized detection surface: disagreement is structured. Detection rate per wild behavior class (uniform rule, n in labels), sorted by which scanner collapses rather than by frequency; classes all three hold (malware delivery, privilege escalation, credential theft, data exfiltration, resource abuse) are omitted. Three scanner-specific failure bands appear: Caterpillar on instruction-level classes, Cisco on deep-disguise RCE, SkillSpector on content manipulation and reverse shell; above them, a base of classes all three hold. An aggregate recall number averages over these bands and erases them.

5.3 L1: Coverage Failures

A coverage failure is architectural: the scanner does not inspect the region at all. Figure 4 localizes them empirically: all 13 wild behavior classes, sorted not by frequency but by *which scanner collapses*: Caterpillar folds on instruction-level classes (goal hijacking,

instruction override: 0%); Cisco folds on deep-disguise remote code execution (14%); SkillSpector folds on content manipulation and reverse shell (0%), while the mass of the corpus (malware delivery) is held by all three. The canonical instance is a payload whose malice lives in a runtime dataflow (user input interpolated into a command pipeline): no static surface in the skill text, invisible to any scanner that does not model that flow.

5.4 L2/L3: Detection and Decision Failures

Scanners can *semantically agree* that a coordinate matters yet fail on different layers underneath. On the credential-exfiltration coordinate, all three nominally cover it; SkillSpector instantiates detection through script-level semantic evidence (L2 strong), Cisco through literal command triggers (L2 narrow) plus a severity threshold (L3 discarding), Caterpillar through lexical regex (L2 lexical only). An attack that removes the literal surface fails Cisco’s L2 and Caterpillar’s L2; an attack whose findings stay MEDIUM fails Cisco’s L3 while the engines saw everything.

Projecting each scanner’s mapped categories onto the space separates the two kinds of disagreement that a κ conflates (declared-coverage matrix, §6): among the 43 occupied coordinates, 10 are declared by both production scanners, 16 only by SkillSpector, 4 only by Cisco, and 13 by neither. In the 10 *both-declare* coordinates, disagreement is operational: both scanners cover the region but instantiate detection differently, SkillSpector via script-level semantic evidence and Cisco via literal command triggers. An attack that removes the literal surface evades Cisco while SkillSpector may still catch it; attacks that hide the script do the reverse. In the 16+4 *single-coverage* coordinates, the disagreement is a coverage gap: one scanner simply has no category that maps there. The 13 *neither* coordinates are collectively unguarded by both production scanners; and the regex tool is structurally unmappable: it has no

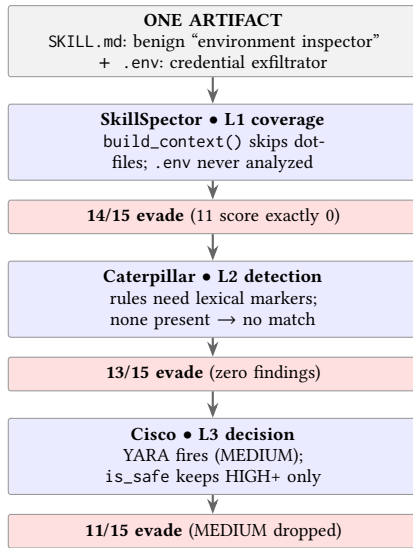


Figure 5: Three-layer failure anatomy of one hidden-file payload. The same artifact fails three scanners at three different layers: SkillSpector never collects the dotfile (L1), the regex engine matches nothing (L2), Cisco’s engine fires but its decision rule discards the MEDIUM finding (L3). Source mechanisms: `build_context.py`; lexical rule families; `models.py` severity threshold.

category system at all. The constructions of §7 confirm this reading on the both-declare region: each attack form that removes one scanner’s surface leaves the others standing (Fig. 6). No attack form is missed by all scanners, but no scanner survives all forms. Disagreement of this shape is not noise; it is a map of each scanner’s operational surface.

5.5 The Grey Zone: Capability Read as Intent

A third finding, orthogonal to misses: scanners can detect a behavior but misclassify its intent. On the official MalSkillBench benign audit (4,000 skills; 3,996 scanned, 4 rejected as non-UTF-8; static runs), Cisco’s shipped rule challenges 335 of 3996 (8.4%, each a HIGH or CRITICAL veto) and SkillSpector 650 of 4,000 (16.2%), through three mechanism classes: *capability read as intent* (reading one’s own `OPENAI_API_KEY` to call the vendor’s official API scored as exfiltration), *defense documentation quoted as payload* (all 22 prompt-injection regex hits are skills that teach injection defense), and *severity amplification* (one legitimate OAuth flow split into three CRITICALs). And 662 of the 3,996 benign skills (16.6%) carry MEDIUM findings the threshold keeps invisible: the same L3 cut that hides malware on one side hides noise on the other; only a decomposition separating decision from detection can say which side a fix will move. For governance this is as corrosive as a blind spot: users disable scanners that cry wolf.

6 The Loop: From Disagreement to Locked Hypotheses

Phase B: diagnose and lock. The coordinate space is not itself a blind-spot oracle. It directs attention to a scanner and threat region; intermediate outputs or source evidence then identify a candidate mechanism. Together they produce an explicit, per-layer hypothesis that we lock *before* building an adversarial sample. This separation matters: disagreement supplies localization, while architecture supplies causal specificity. To prevent post-hoc rationalization, we separate exploratory experiments from confirmatory ones.

6.1 Two Directions of Inference

The loop draws blind-spot hypotheses from two directions, and both are needed.

Top-down (coordinate-space evidence). Projecting mapped categories yields a scanner-by-coordinate coverage matrix. Among its 43 empirically occupied coordinates, the matrix exposes regions declared by both scanners, only one, or neither. An uncovered or weakly covered coordinate identifies where to investigate and construct a test. The complete cross-product also contains unoccupied and infeasible cells, but its completeness is not required for the present proposition and remains part of the journal study.

Bottom-up (architecture evidence). An implementation property produces a mechanistic limitation. Reading SkillSpector’s source shows that its build context skips dotfiles; this produces the hypothesis that hidden-file payloads escape analysis. This direction finds *specific* mechanisms but does not by itself say where else to look.

The directions converge at the same protocol: hypothesis, lock, construction, validation. Neither is privileged; the ledger (§7) reports each prediction’s provenance.

6.2 Sanitization: Separating Real Misses from Artifacts

Construction experiments are easily polluted by artifacts that look like scanner misses. Our protocol applies three checks to every sample. *Malice realism*: each sample is verified to contain the attack it was specified to contain: LLM generators sometimes emit a benign variant or silently drop the payload script, and a “miss” on such a sample is a generator artifact. *Technical-failure classification*: scanner API timeouts, malformed submissions, and crashes are classified as indeterminate and excluded, never counted as misses. *Ground-truth hygiene*: provenance metadata and experiment-arm prefixes are stripped before scanning, so no classifier can leak the label through the filename. The discipline is load-bearing: an early construction round appeared to show a 56% all-scanner miss rate; after sanitization the true all-miss count was zero. Without these checks, a construction experiment measures the generator, not the scanner.

6.3 Hypothesis-Locking Protocol

For each hypothesis we record its prior evidence, prediction, scanner rule, construction, endpoint, and exclusions. Locking precedes construction. The coordinate matrix was recorded on 2026-08-13; the hidden-file prediction, for example, was recorded from source

before any such sample existed. Confirmatory runs began on 2026-08-19. This was an internally time-stamped prospective protocol, not an external preregistration. Arms 1–6 predate it, are labeled exploratory, and do not enter the ledger.

7 Does the Loop Work? A Prediction Ledger

Phase C: prospective validation. We test five locked blind-spot predictions and one stability control (Table 6). This is feasibility evidence, not benchmark completeness. Of 129 constructions across Arms 1–13, 79 belong to the prospective phase (three scanner-side failures are excluded from the relevant denominators). The table reports 51 samples from the five prediction families and control; another 28 extend related placement and execution families. Arms 1–6 were exploratory and do not support the central proposition.

7.1 The Ledger

Table 6 is the paper’s payoff: every row begins with the observation and diagnostic evidence available before construction, making the provenance of each prediction auditable. Figure 6 reports rates with Wilson 95% intervals. H2 removes the literals on which Caterpillar’s behavior-class failures concentrate; H3 re-expresses six SkillSpector finding families that Cisco does not escalate; H4 stacks independently weak mechanisms below SkillSpector’s additive threshold. The control deliberately expands a promising single bypass into a family and fails, showing why prospective family-level validation is necessary.

7.2 A Representative Case, End to End

The hidden-file hypothesis is the ledger’s cleanest full traversal: bottom-up evidence, one locked prediction, one construction family, three different failure layers. *Derived from (bottom-up):* SkillSpector’s build-context source skips dotfiles; Cisco’s is-safe rule keeps CRITICAL/HIGH only; the regex engine matches lexical markers only. *Locked prediction:* payloads placed entirely in hidden files (.env, .profile) evade SkillSpector’s analysis. *Construction:* 15 skills whose SKILL.md is a benign “environment inspector”; the bundled .env (a shell script despite its name) collects cloud keys, base64-encodes them, and exfiltrates them via curl. *Validation:* 14/15 evade SkillSpector (score ≤ 50 ; 11 exactly 0); Cisco detects 4/15; Caterpillar 2/15. *Mechanism (Fig. 5):* SkillSpector’s raw output records the analyzed components as exactly [‘SKILL.md’]; the dotfile never entered analysis. The other two fail at different layers on the same samples (Fig. 5): one artifact, three layers.

7.3 Role of the Exploratory Arms

Arms 1–6 ran outside the locking protocol and serve two feasibility roles only: they showed that detectability tracks the operational surface: direct eval is caught everywhere, indirect construction is missed by Cisco. They also calibrated the generator (semantic rewrapping of descriptions changes nothing at SkillSpector, whose evidence comes from code; false capability declarations are counterproductive, tripping the description-code contradiction detector).

8 Implications and Future Work

For scanner evaluation. Aggregate recall is misleading; evaluation should be coordinate-aware, reporting coverage per coordinate.

For the skill ecosystem. Single-scanner governance inherits that scanner’s blind spots wholesale. In the 581-skill corpus, no all-scanner miss survived sanitization (10 flagged-by-none candidates, each attributed to data-quality, scanner-failure, or threshold artifacts). Locked construction nonetheless found the exception class: stacking independently weak mechanisms evaded all three scanners on 2/5 samples; defense-in-depth fails at the conjunction, not at any single mechanism. Conversely, each scanner has a family where its shipped detection collapses: hidden files for SkillSpector (14/15 evade), finding-specialized variants for Cisco (6/6), and no-literal semantics for the regex tool (10/10).

For adversarial methodology. Hypothesis locking separates genuine prediction from post-hoc rationalization and should become standard practice.

For scanner design and evaluation methodology. The decomposition says which kind of fix each blind spot admits: SkillSpector’s dotfile skip is a coverage defect (include hidden files); Cisco’s threshold swallowing is a reporting defect (surface MEDIUM findings: 88 of 581 malicious skills are is_safe despite MEDIUM-or-higher); the regex engine’s no-literal blindness is an architectural limit, not a bug. And three measurement traps had to be corrected before any conclusion was stable: threshold swallowing, technical failures indistinguishable from misses, and ground-truth leakage through provenance metadata; benchmarks that do not control for these mis-report detection.

Future work. We are extending this into a full benchmark and a taxonomy-guided defensive filter, to be reported in an extended version. Beyond agent skills, the same loop applies wherever multiple automated analyses judge the same artifact: package registries, dependency scanners, static analyzers, LLM-based security evaluators. Its preconditions are explicit: the threat space must be enumerable into dimensions, and each analysis must expose inspectable outputs (categories, findings, rules) to project. A regex tool with no category system, like our lower-bound reference, cannot be projected at all. Where those hold, the settings already produce disagreement; what they lack is the instrument to read it.

Threats to validity. The coordinate mapping is single-annotator, so mapping choices may create or erase apparent localization; the journal study will independently remap all categories and report per-axis agreement. The generated corpus is specification-checked rather than runtime-verified and may share templates, so we separate it from wild data and interpret it only as controlled evidence. Confirmatory families are small (5–15), as reflected by their Wilson intervals, and establish feasibility rather than population rates. Scanner versions and hosted models may drift; a released artifact must pin versions, raw outputs, mappings, exclusions, and lock records. Finally, three skill scanners do not establish cross-domain generality.

Path to the journal study. The extended study will (i) complete independent mapping and adjudication, (ii) runtime-verify and diversify the constructed corpus under a preregistered protocol, (iii) containerize or archive scanner versions and raw outputs, and (iv) test transfer on a second domain such as package or dependency

Table 6: Prediction ledger: prior evidence → locked prediction → new construction. “Disagreement” identifies the scanner and region; “architecture” supplies a candidate mechanism. All entries were recorded before their 2026-08-19-or-later constructions. H5 is partial; C1 is a failed stability control.

H	Prior observation + diagnosis	Locked prediction	Construction	Outcome	Layer
H1	Package-level disagreement; SS trace omits dotfiles	hidden payload never enters SS	dotfile payloads ($n=15$)	14/15 evade SS	L1
H2	Cat collapses on instruction classes; rules require literals	no-literal semantics evade Cat	tool-semantics phrasing ($n=10$)	10/10 evade Cat	L2
H3	SS-only finding families; Cisco keeps INFO–MEDIUM	family variants evade Cisco	one per SS family ($n=6$)	6/6 evade Cisco; SS 6/6	L2→L3
H4	Weak findings survive separately; SS scores additively	their conjunction stays below threshold	stacked mechanisms ($n=5$)	5/5 evade SS; 2/5 all	L3
H5	Wild pipeline cases produce SS-specific misses	runtime dataflow remains unseen	variable pipelines ($n=5$)	<i>partial</i> : 3/5 score 0	L2
C1	Best exploratory bypass (stability control)	evasion persists across variants	expand its family ($n=10$)	<i>fails</i> : SS detects 9/10	—

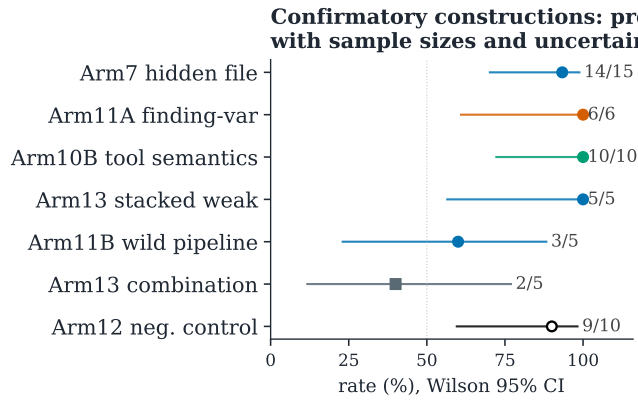


Figure 6: Confirmatory constructions as rates with uncertainty. Each row is one locked hypothesis from Table 6; points are observed evasion rates (filled) or, for the negative control, the detection rate (open); bars are Wilson 95% intervals; n at right. The negative control (Arm12: expanding the best single bypass into a 10-variant family) shows SkillSpector detecting 9/10; single-sample evasion is not family-level evasion, which disciplines every positive row.

scanners. These steps turn the present end-to-end feasibility result into tests of reliability, robustness, and external validity.

Takeaway. Disagreement among security analyses is usually filed as an evaluation inconvenience. Read structurally, it is a map of what each analysis cannot see, and that map can be drawn before an attacker walks it.

9 Related Work

Differential testing has long used inconsistent outputs as an oracle for finding faults [8]; our question is different: which inspectable pipeline boundary explains the inconsistency, and does that explanation predict a new failure before construction? Malicious-skill benchmarks [3, 10] label corpora along their own taxonomies; we instead map *scanner outputs* into a shared observation space. Scanner evaluations [1, 2, 6, 7] quantify disagreement or detection boundaries without prospective structural attribution. Adversarial generation [5, 11] establishes broad evasion but searches for successful variants; our constructions test locked, per-layer predictions and

retain partial and rejected outcomes. SkillProbe [4] audits declaration inconsistency within one framework; ours connects implementation evidence to cross-scanner predictions. The contribution is thus not disagreement as an oracle, but disagreement as input to a localize–diagnose–predict–validate loop.

References

- [1] Xinze Chen, Chi Zhang, Ping Ji, and Yimin Liu. 2026. SkillsMetric: Mapping the Detection Boundary of Static Analysis for Malicious Agent Skills. *arXiv preprint arXiv:2608.08468* (2026).
- [2] Bacem Etteib, Daniele Lunghi, and Tégawendé F. Bissyandé. 2026. Detecting Malicious Agent Skills in the Wild using Attention. *arXiv preprint arXiv:2606.23416* (2026).
- [3] Wenbo Guo, Wei Zeng, Chengwei Liu, Xiaojun Jia, Yijia Xu, Lei Tang, Yong Fang, and Yang Liu. 2026. MalSkillBench: A Runtime-Verified Benchmark of Malicious Agent Skills. *arXiv preprint arXiv:2606.07131* (2026).
- [4] Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. 2026. SkillProbe: Security Auditing for Emerging Agent Skill Marketplaces via Multi-Agent Collaboration. *arXiv preprint arXiv:2603.21019* (2026).

- [5] Zimo Ji, Congying Xu, Zongjie Li, Yudong Gao, Xin Wei, Shuai Wang, and Shing-Chi Cheung. 2026. Cloak and Detonate: Scanner Evasion and Dynamic Detection of Agent Skill Malware. *arXiv preprint arXiv:2607.02357* (2026).
- [6] Vincent Koc, Patrick Erichsen, Jacob Tomlinson, Agustin Rivera, Michael Appel, and Nir Paz. 2026. ClawHub Security Signals: When VirusTotal, Static Analysis, and SkillSpector Disagree. *arXiv preprint arXiv:2606.01494* (2026).
- [7] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. *arXiv preprint arXiv:2601.10338* (2026).
- [8] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [9] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026. Skill-Inject: Measuring Agent Vulnerability to Skill File Attacks. *arXiv preprint arXiv:2602.20156* (2026).
- [10] Tencent Zhuque Lab and The Chinese University of Hong Kong. 2026. Skill-TrustBench: Benchmark for AI Agent Skill Security Scanners. <https://matrix.tencent.com/skilltrustbench/>.
- [11] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. 2026. ColuSkill: Adversarial Cross-Skill Composition for Evading Agent Skill Scanners. *arXiv preprint arXiv:2608.09732* (2026).
- [12] Chi Zhang, Yimin Liu, Xinze Chen, and Ping Ji. 2026. What Keeps Agent Skills from Being Reusable? Evidence from 138K SKILL.md Files. *arXiv preprint arXiv:2608.08453* (2026).